SCE 672 – Assessment 2: Flatness Pursuit and Motor Voltage Dynamics

Planned Path, Command Path, Reference Profiles and Tracking Errors (Target: Tag 3)

Faisal Zahid – g202530490 KFUPM

October 2026

1 Planned path (from the `rm_flatness_nav` code)

The flat output is $z(t) = [x(t), y(t)]^\top$, and each component is a cubic $z_j(t) = \sum_{k=0}^{3} \alpha_k^{[j]} t^k$. The coefficients come from the boundary-condition system $M\alpha^{[j]} = [z_j(0), \dot{z}_j(0), z_j(T), \dot{z}_j(T)]^\top$ (Eq. 2 of the brief), solved with `numpy.linalg.solve`. The flatness inversion recovers

$$v = \sqrt{\dot{x}^2 + \dot{y}^2}, \qquad \omega = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}. \tag{1}$$

Listing 1: `trajectory.py`: cubic solve and plan construction.

```
def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    """Solve the four cubic coefficients from start/end position and speed."""
    if T <= 0:
        raise ValueError("T must be positive")

    matrix = np.array([
        [1, 0, 0, 0],
        [0, 1, 0, 0],
        [1, T, T*T, T*T*T],
        [0, 1, 2*T, 3*T*T],
    ], dtype=float)
    boundary = np.array([z0, zdot0, zf, zdotf], dtype=float)
    return np.linalg.solve(matrix, boundary)


def make_trajectory(d0):
    """Plan from the current distance d0 to a 0.20 m standoff."""
    if d0 <= STOP_DISTANCE_M:
        raise ValueError("Already at the stopping distance")

    travel = d0 - STOP_DISTANCE_M
    T = max(1.0, 1.6 * travel / MAX_SPEED_MPS)

    alpha_x = solve_flat_polynomial(0.0, 0.06, travel, 0.0, T)
    alpha_y = solve_flat_polynomial(0.0, 0.0, 0.0, 0.0, T)
    return T, alpha_x, alpha_y
```

Listing 2: `trajectory.py`: `evaluate()` implements the inversion.

```
def evaluate(T, alpha_x, alpha_y, t):
    """Return planned x, y, linear speed v, and turn rate omega."""
    t = min(max(float(t), 0.0), T)

    def position_velocity_acceleration(a):
        position = a[0] + a[1]*t + a[2]*t*t + a[3]*t*t*t
        velocity = a[1] + 2*a[2]*t + 3*a[3]*t*t
        acceleration = 2*a[2] + 6*a[3]*t
        return position, velocity, acceleration

    x, xdot, xddot = position_velocity_acceleration(alpha_x)
    y, ydot, yddot = position_velocity_acceleration(alpha_y)

    v = float(np.hypot(xdot, ydot))
    speed_squared = xdot*xdot + ydot*ydot
    omega = (
        float((xdot*yddot - ydot*xddot) / speed_squared)
        if speed_squared > 1e-12 else 0.0
    )

    if t >= T:
        v, omega = 0.0, 0.0
    return float(x), float(y), v, omega
```

Parameters of the executed plan. d_0 is the camera distance at the moment the plan was created. It was not logged, so I recovered it by fitting the plan's $v(t)$ to the six logged CHASING commands (two free parameters: d_0 and the plan start time). The fit residual is below 5×10^{-4} m/s on every sample.

Quantity	Value	Source
d_0	1.435 m	fitted from CSV
Travel $d_0 - 0.20$	1.235 m	code
$T = \max(1.0,\ 1.6\,\text{travel}/v_{max})$	12.35 s	code, $v_{max} = 0.16$ m/s
Plan start (log time)	$t \approx 1.19$ s	fitted
α_x	$[0,\ 0.06,\ 0.014575,\ -0.000918]$	`solve_flat_polynomial`
α_y	$[0, 0, 0, 0]$	$y(0) = \dot{y}(0) = y(T) = \dot{y}(T) = 0$

Since $\alpha_y \equiv 0$, the planned path is a straight line along body $+x$ with $\omega_{ref} \equiv 0$. Boundary check: $x(T) = 1.235$ m and $\dot{x}(T) = 0$.

Deviation from the brief: the code uses a floor of 1.0 s in $T = \max(\cdot)$, while the brief specifies 4.0 s. The floor is inactive here ($T = 12.35$ s > 4 s), so the executed plan is the same either way.

2 Where the plan is commanded to the RoboMaster

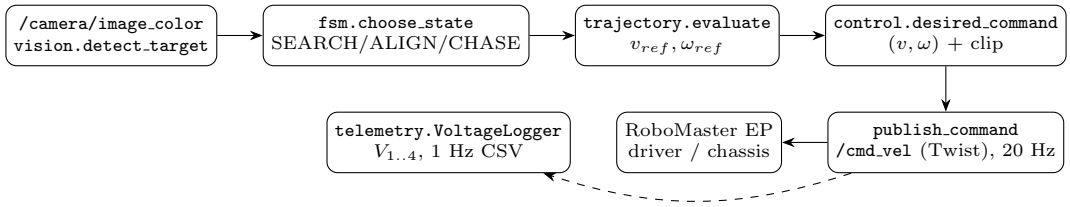


Figure 1: Command path. The dashed arrow means the logger receives the *commanded* (v, ω).

The planner's 20 Hz timer (`create_timer(0.05, update_state)`, line 124) calls `desired_command()` and then `publish_command()` (line 437). The `/cmd_vel` publisher exists only when `enable_motion:=true` (line 95). The RoboMaster driver subscribes to `/cmd_vel`; the driver is not part of the uploaded package, so I do not list it.

Listing 3: `planner.py`: the only place a command leaves the planner.

```
222     def publish_command(self, v, yaw):
223         if self.cmd_pub is None:
224             return
225         command = Twist()
226         command.linear.x = float(v)
227         command.angular.z = float(yaw)
228         self.cmd_pub.publish(command)
```

Listing 4: `control.py`: CHASING branch maps the plan to (v, ω). Because $\omega_{ref} = 0$, the yaw output is pure bearing feedback, $\omega = -0.8\,\theta_{bearing}$.

```
68      if state == State.CHASING and plan is not None:
69          T, alpha_x, alpha_y = plan
70          if plan_time_s >= T:
71              return 0.0, 0.0
72
73          _, _, planned_v, planned_yaw = evaluate(
74              T, alpha_x, alpha_y, plan_time_s
75          )
76          yaw = planned_yaw - 0.8 * error
77          yaw = max(-MAX_TURN_RPS, min(MAX_TURN_RPS, yaw))
78          forward = max(0.0, min(MAX_FORWARD_MPS, planned_v))
79          return forward, yaw
```

Voltages are model outputs, not measurements. `motor_model.py` maps the commanded (v, ω) to wheel rates ω_i (Eq. 4) and then to $V_i = K_v \omega_i + K_a \dot{\omega}_i$ (Eq. 5), with $K_v = 0.38$ and $K_a = 0.06$.

3 Reference profiles: velocity and x

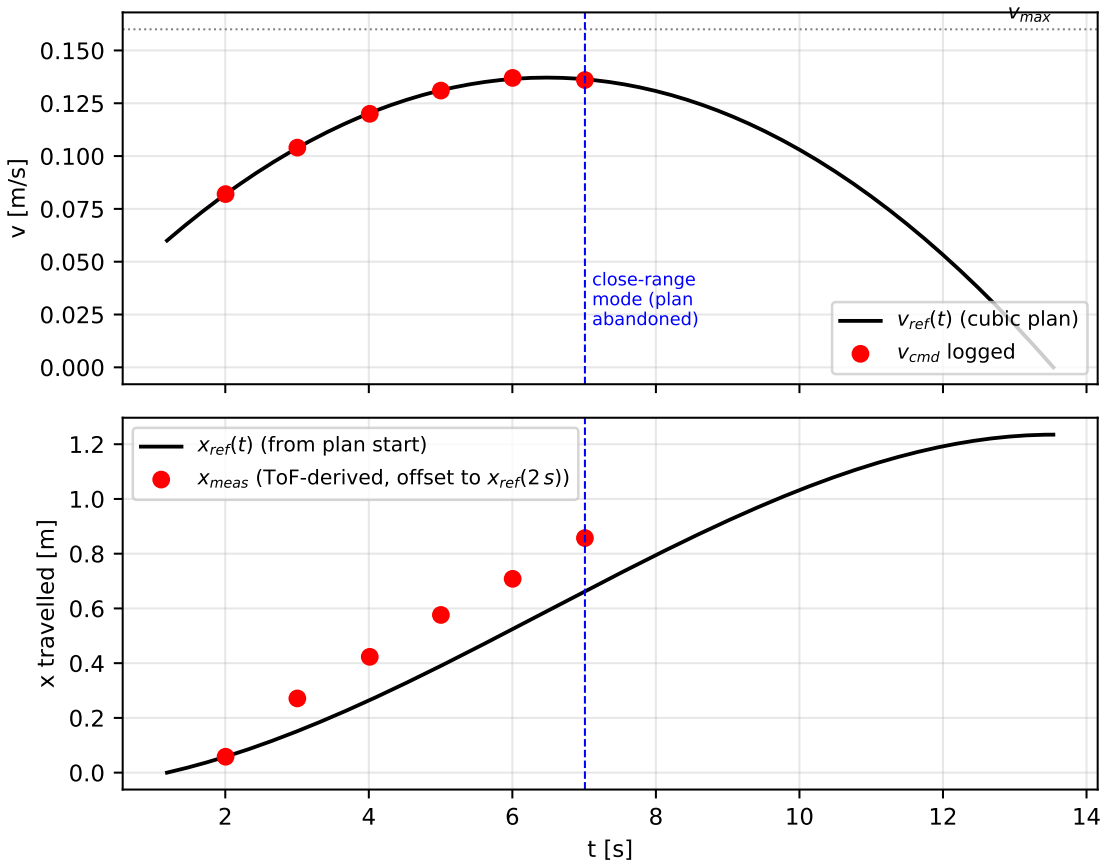


Figure 2: Planned $v_{ref}(t)$ and $x_{ref}(t)$ (full 12.35 s cubic, placed on log time) against the logged command and the ToF-derived progress. The plan peaks at 0.137 m/s, below v_{max}. It was abandoned at $t \approx 7$ s when the camera range fell below 0.35 m and the close-range controller took over.

4 Tracking errors

Only commands, ToF and state were logged; there is no `/odom` record. The three error signals below are therefore limited as stated.

- $e_v = v_{cmd} - v_{ref}$ checks that the logged commands follow the plan. It is nearly zero by construction, because d_0 and the start time were fitted to these same samples. It is not a measurement of wheel speed.
- $e_x = x_{meas} - x_{ref}$ uses the ToF change since $t = 2.0$ s as measured progress. This is a proxy only: the ToF reads the nearest obstacle, not the tag (see `control.py` comments).
- Bearing error is recovered from the yaw command, $\theta = -\omega_{cmd}/0.8$, valid because $\omega_{ref} = 0$ and the command is unclipped.

Table 1: Tracking errors over the executed CHASING segment (1 s samples). x in m, e_v in mm/s.

t [s]	v_{ref}	v_{cmd}	e_v	x_{ref}	x_{meas}	e_x	bearing [deg]
2.0	0.082	0.082	+0.05	0.000	0.000	+0.000	-3.87
3.0	0.104	0.104	+0.14	0.093	0.213	+0.120	-1.72
4.0	0.120	0.120	-0.37	0.207	0.365	+0.158	-0.86
5.0	0.131	0.131	-0.14	0.332	0.518	+0.186	-0.21
6.0	0.137	0.137	+0.48	0.466	0.650	+0.184	-0.43
7.0	0.136	0.136	-0.37	0.604	0.799	+0.195	-0.86

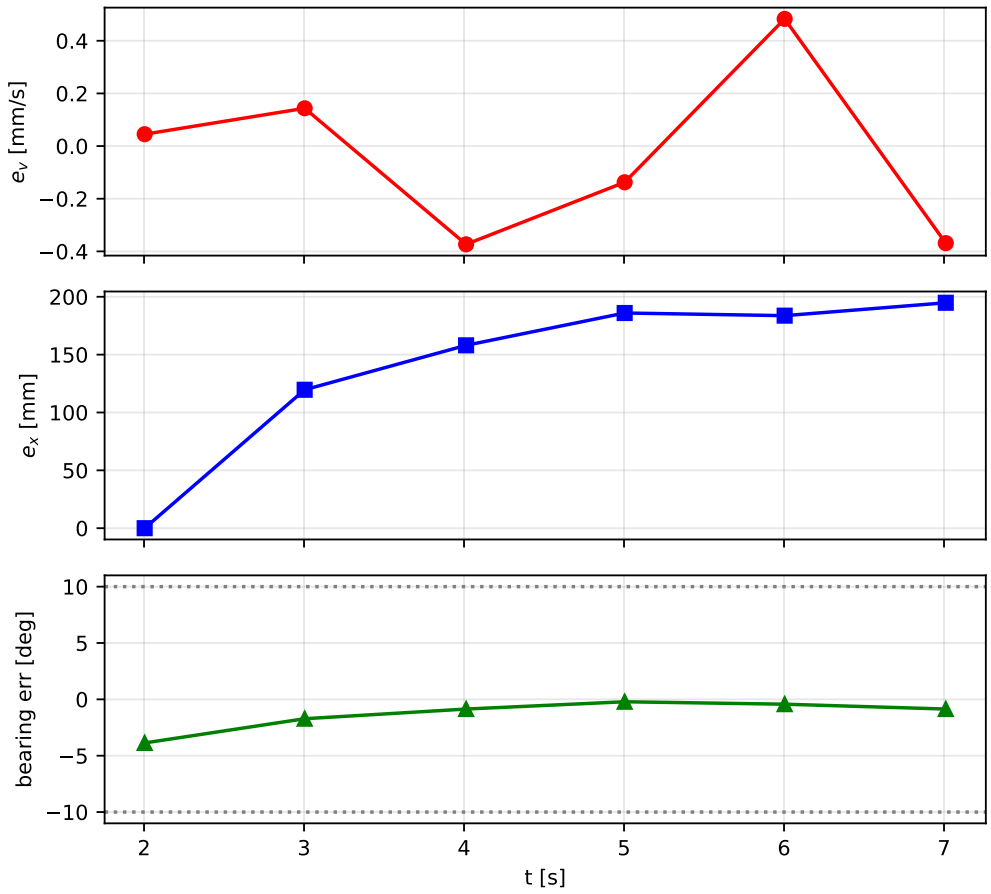


Figure 3: e_v, e_x and bearing error. The dotted lines mark the $\pm 10°$ alignment limit.

Results.

- Command vs. plan: RMS $e_v = 0.30$ mm/s, max 0.48 mm/s (consistency check only).
- Heading: the bearing error is at most 3.9° (0.068 rad), well inside the 10° limit, and decays toward 0°.
- Progress: the ToF-derived progress runs ahead of the plan. Over 5 s the ToF fell by 0.799 m, a mean of 0.160 m/s, while the mean commanded speed was 0.121 m/s. This gives e_x of about 0.19 m (RMS 0.16 m), and the ToF-derived speed would even exceed v_{max}. The log alone cannot say whether this comes from the ToF geometry or from the chassis moving faster than commanded. A proper position-tracking error needs the `/odom` record.

5 Telemetry and halt (Table 1)

Table 2: Modeled motor voltages, `motor_voltage_log.csv` (Tag 3).

t [s]	V_1 [V]	V_2 [V]	V_3 [V]	V_4 [V]	State
0.0	0.000	0.000	0.000	0.000	SEARCHING
1.0	0.528	-0.528	-0.528	0.528	ALIGNING
2.0	0.742	0.695	0.695	0.742	CHASING
3.0	0.846	0.788	0.788	0.846	CHASING
4.0	0.949	0.918	0.918	0.949	CHASING
5.0	1.013	1.009	1.009	1.013	CHASING
6.0	1.055	1.034	1.034	1.055	CHASING
7.0	1.055	1.016	1.016	1.055	CHASING
8.0	-0.168	-0.162	-0.162	-0.168	CHASING
9.0	0.000	0.000	0.000	0.000	SEARCHING
10.0	0.000	0.000	0.000	0.000	CHASING
11.0	0.000	0.000	0.000	0.000	SEARCHING
12.0	0.000	0.000	0.000	0.000	SEARCHING
13.0	0.519	0.470	0.470	0.519	CHASING
14.0	-0.070	-0.064	-0.064	-0.070	SEARCHING
15.0	0.000	0.000	0.000	0.000	SEARCHING
16.0	0.000	0.000	0.000	0.000	SEARCHING
17.0	0.000	0.000	0.000	0.000	ARRIVED
18.0	0.000	0.000	0.000	0.000	ARRIVED
19.0	0.000	0.000	0.000	0.000	ARRIVED
20.0	0.000	0.000	0.000	0.000	ARRIVED
21.0	0.000	0.000	0.000	0.000	ARRIVED
22.0	0.000	0.000	0.000	0.000	ARRIVED
23.0	0.000	0.000	0.000	0.000	ARRIVED

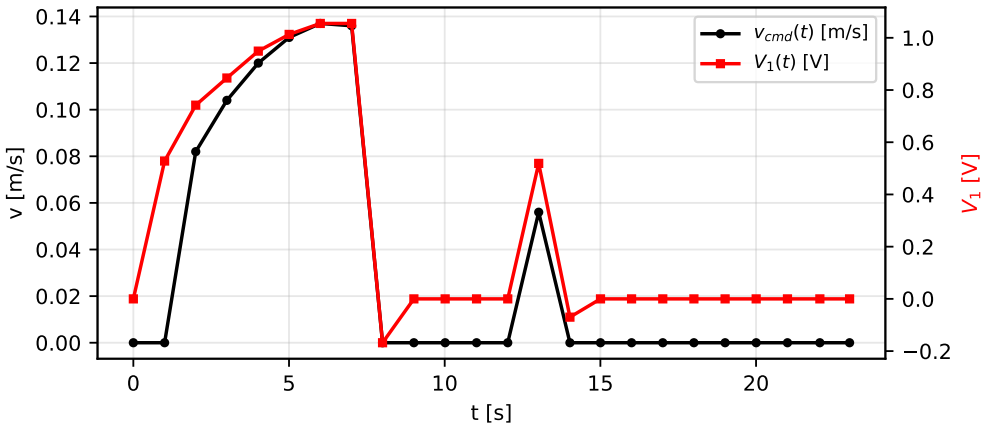


Figure 4: Commanded forward speed $v(t)$ and modeled $V_1(t)$.

The voltages are exactly 0.000 V in the ARRIVED state ($t = 17$–23 s), so the halt is confirmed. ARRIVED is triggered by the camera range ≤ 0.20 m, and the ToF then reads a steady 0.225–0.228 m, so it is not referenced to the same point as the camera. The negative V values at $t = 8$ s (-0.168 V) and $t = 14$ s (-0.070 V) are the $K_a\dot{\omega}$ deceleration term when the command drops to zero. The SEARCHING/CHASING flicker over $t = 9$–16 s occurs at the close-range tag-loss stage, with $v = 0$ except at $t = 13$ s.